DuckDB-Based Airline Market Analysis Dashboard

Kyung Je Park

USC# 4030598098

DSCI 551 – Foundation of Data Management

University of Southern California

Professor Wu

5/8/2026

1. Introduction & Motivation

This project explores the internal architecture of DuckDB and applies it to an analytical application. The application is a Streamlit-based airline market analysis dashboard that examines airline pricing, passenger demand, and market concentration across U.S. routes.

The dataset contains airline route and fare data from 1993 to 2024, including attributes such as origin and destination cities, passenger volume, average fares, and carrier market share. This makes it suitable for analyzing how competition influences pricing behavior.

The main motivation of this project is to understand how database internals impact application performance and behavior. Instead of only building a dashboard, the project focuses on how DuckDB processes analytical queries internally. In particular, it examines how columnar storage and vectorized execution support aggregation-heavy workloads.

The application implements three core operations: *yearly fare trend analysis*, *route-level market summary*, and *high-concentration route filtering*. Each operation is connected to DuckDB's internal execution using EXPLAIN, allowing a direct mapping between application behavior and database internals.

2. Database System Overview

DuckDB is an embedded analytical database management system designed for local data analysis and OLAP workloads. It can run directly inside a Python process without requiring a separate database server. This makes it convenient for data science and analytical applications because users can load data, execute SQL queries, and integrate results with Python tools.

In this project, DuckDB is used as the backend database engine. The application is written in Python, and the user interface is built using Streamlit. The dataset is loaded into a DuckDB database file named, *us_airline_data.duckdb* and the main analytical table is *flights*. This table is created from the airline CSV dataset using a setup script. The core schema includes the following important attributes: *Year, quarter, city1, city2, airport_1, airport_2, passengers, fare, carrier_lg, large_ms, fare_lg, carrier_low, lf_ms, fare_low*.

The project uses a single-table analytical design. This is appropriate because the main goal is not transactional normalization, but efficient analytical querying. The dashboard continuously scans, filters, groups, and summarizes the airline data.

DuckDB is a good fit for this application because the workload is mainly read-oriented. The dashboard does not frequently insert, update, or delete records. Instead, it repeatedly asks analytical questions such as:

- What is the average fare by year?
- Which routes have the highest passenger volume?
- Which routes are highly concentrated under one dominant carrier?
- How large is the fare gap between the dominant carrier and the lowest-fare carrier?

These are analytical queries, not transactional operations. Therefore, DuckDB's design is more suitable than a traditional OLTP-oriented system.

3. Internal Architecture

The internal architecture of DuckDB plays a central role in this project, particularly because the application is designed for analytical workloads. The key architectural components examined in this project are columnar storage, vectorized query execution, and query execution planning with optimization techniques.

DuckDB adopts a columnar storage model, where data is stored by column rather than by row. This design is particularly advantageous for analytical queries, which typically access only a subset of columns while scanning a large number of rows. For example, the yearly fare trend query only requires attributes such as year, fare, fare gap, and passenger count. Instead of reading the entire row, DuckDB scans only the necessary columns. This behavior is visible in the EXPLAIN plan, where the *SEQ_SCAN* operator lists only the projected columns (see Appendix B). By avoiding unnecessary column access, columnar storage significantly reduces I/O overhead and improves query performance.

Another important feature of DuckDB is its vectorized execution engine. Rather than processing one row at a time, DuckDB operates on batches (vectors) of values. This allows operations such as filtering, arithmetic computation, and aggregation to be applied efficiently across many rows at once. In this project, computations such as *fare_lg - fare_low* and aggregate functions like *AVG* and *SUM* benefit from vectorized execution. This approach improves CPU efficiency, reduces function call overhead, and enables better cache utilization.

DuckDB also provides detailed insight into its execution through the EXPLAIN command, which reveals how queries are processed internally. The execution plan is composed of operators such as *SEQ_SCAN, FILTER, PROJECTION, PERFECT_HASH_GROUP_BY, ORDER_BY*, and *TOP_N*. Each operator corresponds to a logical step in query execution. For example, *SEQ_SCAN* represents data access, *FILTER* applies conditions, *PROJECTION* computes expressions, and *PERFECT_HASH_GROUP_BY* performs aggregation.

In more complex queries, DuckDB applies query optimization techniques. One example in this project is the use of the TOP_N operator in the high-concentration routes query. Instead of

sorting the entire dataset, DuckDB identifies only the top results based on the ordering condition. This reduces computational cost and improves performance. Additionally, the execution plan includes a semi join, which allows DuckDB to efficiently retrieve full row data after identifying candidate rows.

Overall, DuckDB's internal architecture is well-suited for analytical applications. The combination of columnar storage, vectorized execution, and optimized query planning enables efficient processing of aggregation-heavy workloads, which directly benefits the performance of the dashboard implemented in this project.

4. Application Design

The application is implemented as a Streamlit dashboard backed by DuckDB. It provides an interactive interface where users can adjust parameters such as year range, number of routes, and market concentration threshold.

The project consists of several components. The *a_schema.sql* and *b_setup_database.py* files are used to create and populate the DuckDB database. The *c_queries.py* file contains reusable SQL queries for each operation. The *d_app.py* file provides a command-line version, while the final interface is implemented in *e_dashboard.py*.

The us*_airline_data.duckdb* file stores the database used by the application, while the CSV file serves as the raw dataset used to create the table.

The dashboard includes three main sections: **yearly fare trend analysis**, **route market summary**, and **high-concentration routes**. It also includes an EXPLAIN section that displays the internal execution plan for each query. A sample dashboard view is shown in Appendix A.

5. Mapping Internals to Application Behavior

5.1 Yearly Fare Trend Analysis

The first operation computes average fare, average fare gap, and total passengers by year. This involves grouping data by year and performing aggregation.

Internally, DuckDB performs a sequential scan of the table while reading only the required columns. It applies filters, computes the fare gap, and performs a hash-based group-by operation. Finally, the results are sorted by year.

This mapping shows how columnar storage reduces unnecessary column access and how vectorized execution improves aggregation efficiency. The EXPLAIN plan is shown in Appendix B.

5.2 Route Market Summary

The second operation summarizes airline data by route, grouping by origin and destination cities. It calculates average market share, average fare gap, and total passengers.

DuckDB scans only relevant columns, applies filtering, computes derived values, and performs a group-by aggregation on route pairs. The results are then sorted and limited.

This demonstrates how DuckDB efficiently handles large-scale aggregation queries. The execution plan for this operation is shown in Appendix C.

5.3 High-Concentration Routes

The third operation filters routes where a dominant carrier exceeds a specified market share threshold and ranks them by fare gap.

The EXPLAIN plan for this operation is more complex. DuckDB applies filtering, computes the fare gap, and uses a TOP-N optimization to identify the highest-ranked rows. It then performs a semi join to retrieve the corresponding full rows.

This shows that DuckDB not only performs basic query execution but also applies optimization techniques to improve performance. The execution plan is shown in Appendix D.

6. Comparison with MySQL and MongoDB

6.1 DuckDB vs MySQL

MySQL is a relational database system commonly used for OLTP (Online Transaction Processing) workloads. It is well-suited for applications that require frequent inserts, updates, deletes, transactions, and point lookups.

However, this project is not primarily transactional. The airline dashboard performs analytical operations such as *GROUP BY*, *AVG*, *SUM*, *ORDER BY*, *LIMIT*, and *filtering by thresholds*. These operations are more aligned with OLAP (Online Analytical Processing) workloads.

A traditional MySQL setup is row-oriented, meaning full rows are stored together. This is efficient when an application needs many columns from a small number of rows. However, for analytical queries that scan many rows but use only a few columns, a row-based storage model may result in unnecessary data reads.

DuckDB is better suited for this project because it is columnar, scans only the required columns, uses vectorized execution, and is optimized for analytical queries. It also integrates easily with Python and local CSV data. For example, the yearly fare trend query requires only a few columns. DuckDB reads only those columns, whereas a row-oriented system may still access entire rows even when most columns are not needed.

This does not mean DuckDB is always better than MySQL. If the application were an airline booking system with frequent customer transactions, ticket purchases, and updates, MySQL would likely be more appropriate. However, for this analytical dashboard, DuckDB is the better fit.

6.2 DuckDB vs MongoDB

MongoDB is a document-oriented NoSQL database that is well-suited for semi-structured, nested, or schema-flexible data. It is commonly used when records vary in structure or when JSON-like document storage is preferred.

In this project, the airline dataset is structured and tabular, with consistent columns such as year, city, fare, passengers, and market share. The queries are also SQL-based analytical operations involving grouping, aggregation, filtering, and sorting. For this reason, DuckDB is a better fit for the workload than MongoDB.

Although MongoDB supports aggregation through its aggregation pipeline, this project benefits more from SQL-based querying and columnar execution. DuckDB also works directly with CSV data and integrates naturally with tools like pandas and Streamlit.

MongoDB would be more appropriate if the dataset were stored as nested documents—for example, records containing detailed flight segments, customer profiles, or booking histories. However, for structured, route-level analytical data, DuckDB provides a more direct and efficient solution.

7. Limitations & Lessons Learned

7.1 Limitations

The first limitation is that the project runs on a single local machine. DuckDB is embedded and local in this implementation. The project does not explore distributed query processing or multi-node execution.

The second limitation is that the application focuses on analytical queries only. It does not support transactional operations such as inserting new flight records, updating fares, or managing concurrent users.

The third limitation is that the project does not provide a full benchmark comparison with MySQL or MongoDB. The project explains why DuckDB is suitable for this workload, but a complete performance benchmark would require loading the same dataset into multiple systems and measuring the same queries repeatedly under controlled conditions.

The fourth limitation is that the dashboard is relatively simple. It provides useful inputs and outputs, but it does not include advanced visualizations, user authentication, or deployment to a production server.

The fifth limitation is that the dataset is treated as a single analytical table. This is acceptable for the project's analytical purpose, but a more complex data warehouse design could separate route, carrier, airport, and fare information into dimension and fact tables.

7.2 Lessons Learned

The most important lesson from this project is that database internals directly affect application behavior. At first, the application looked like a simple Python dashboard that runs SQL queries. However, using EXPLAIN showed that each query maps to specific database operators such as scanning, projection, filtering, grouping, sorting, and Top-N optimization.

Another important lesson is the difference between OLAP and OLTP workloads. This project is OLAP-oriented because it analyzes many rows and computes summaries. DuckDB is a better fit for this type of workload than a traditional transactional database.

The project also helped demonstrate why columnar storage matters. When a query only needs a few columns, DuckDB can avoid scanning unnecessary attributes. This is especially valuable for wide datasets.

The project also showed the importance of vectorized execution. Analytical queries often apply the same operation to many values. Processing those values in batches is more efficient than row-by-row execution.

Finally, the project showed that a simple interface can become much more useful when user inputs are added. The original command-line app proved that the queries worked, but the Streamlit dashboard made the application more interactive and easier to demonstrate.

8. Conclusion

This project developed a DuckDB-based airline analysis dashboard that connects application behavior to database internals. The application supports analytical operations such as aggregation, filtering, and derived metric computation through an interactive Streamlit interface.

DuckDB was chosen because its columnar storage and vectorized execution model align well with this analytical workload. Using EXPLAIN plans, the project demonstrated how queries are processed internally, including column scanning, aggregation, and optimizations such as Top-N and semi joins.

Overall, the project highlights how database internals directly impact real-world analytical applications, and shows that DuckDB is an effective choice for this type of workload.

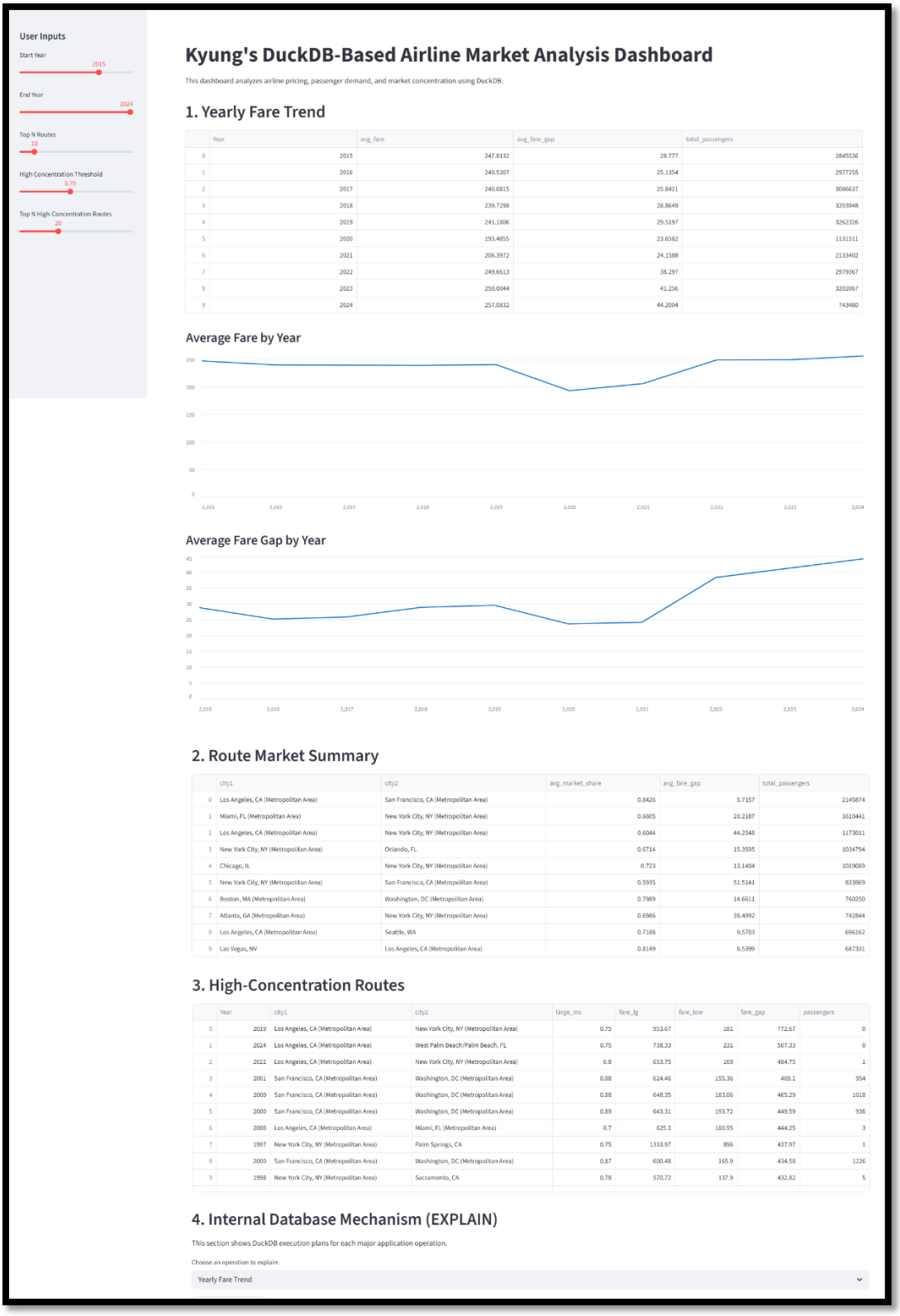
9. Project Repository

The complete project source code, including setup instructions and documentation, is available in the following GitHub repository:

https://github.com/yourusername/duckdb-airline-analysis-dashboard

The repository includes all necessary files to reproduce the application, including source code, dependency configuration, and dataset setup instructions.

Appendix A — Dashboard Interface



	Year	avg_fare	avg_fare_gap	total_passengers
0	2015	247.8132	28.777	2845536
1	2016	240.5307	25.1354	2977255
2	2017	240.0815	25.8421	3086637
3	2018	239.7298	28.8649	3203948
4	2019	241.1806	29.5197	3262326
5	2020	193.4855	23.6582	1131511
6	2021	206.3972	24.1588	2133402
7	2022	249.6613	38.297	2979367
8	2023	250.0044	41.256	3202067
9	2024	257.0832	44.2004	743480

	city1	city2	avg_market_share	avg_fare_gap	total_passengers
0	Los Angeles, CA (Metropolitan Area)	San Francisco, CA (Metropolitan Area)	0.8426	5.7157	2145874
1	Miami, FL (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6605	20.2187	1610441
2	Los Angeles, CA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6044	44.2548	1173011
3	New York City, NY (Metropolitan Area)	Orlando, FL	0.6714	15.3595	1034794
4	Chicago, IL	New York City, NY (Metropolitan Area)	0.723	13.1404	1019089
5	New York City, NY (Metropolitan Area)	San Francisco, CA (Metropolitan Area)	0.5935	51.5141	833969
6	Boston, MA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.7989	14.6611	760250
7	Atlanta, GA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6986	38.4092	742844
8	Los Angeles, CA (Metropolitan Area)	Seattle, WA	0.7188	9.5703	696162
9	Las Vegas, NV	Los Angeles, CA (Metropolitan Area)	0.8149	8.5399	687331

	Year	city1	city2	large_ms	fare_lg	fare_low	fare_gap	passengers
0	2019	Los Angeles, CA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.75	953.67	181	772.67	0
1	2024	Los Angeles, CA (Metropolitan Area)	West Palm Beach/Palm Beach, FL	0.75	738.33	231	507.33	0
2	2022	Los Angeles, CA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.8	653.75	169	484.75	1
3	2001	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.88	624.46	155.36	469.1	954
4	2000	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.88	648.35	183.06	465.29	1018
5	2000	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.89	643.31	193.72	449.59	936
6	2000	Los Angeles, CA (Metropolitan Area)	Miami, FL (Metropolitan Area)	0.7	625.2	180.95	444.25	3
7	1997	New York City, NY (Metropolitan Area)	Palm Springs, CA	0.75	1333.97	896	437.97	1
8	2000	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.87	600.48	165.9	434.58	1226
9	1998	New York City, NY (Metropolitan Area)	Sacramento, CA	0.78	570.72	137.9	432.82	5

Figure A1: Streamlit dashboard showing user inputs and analytical outputs.

Appendix B — EXPLAIN: Yearly Fare Trend Analysis

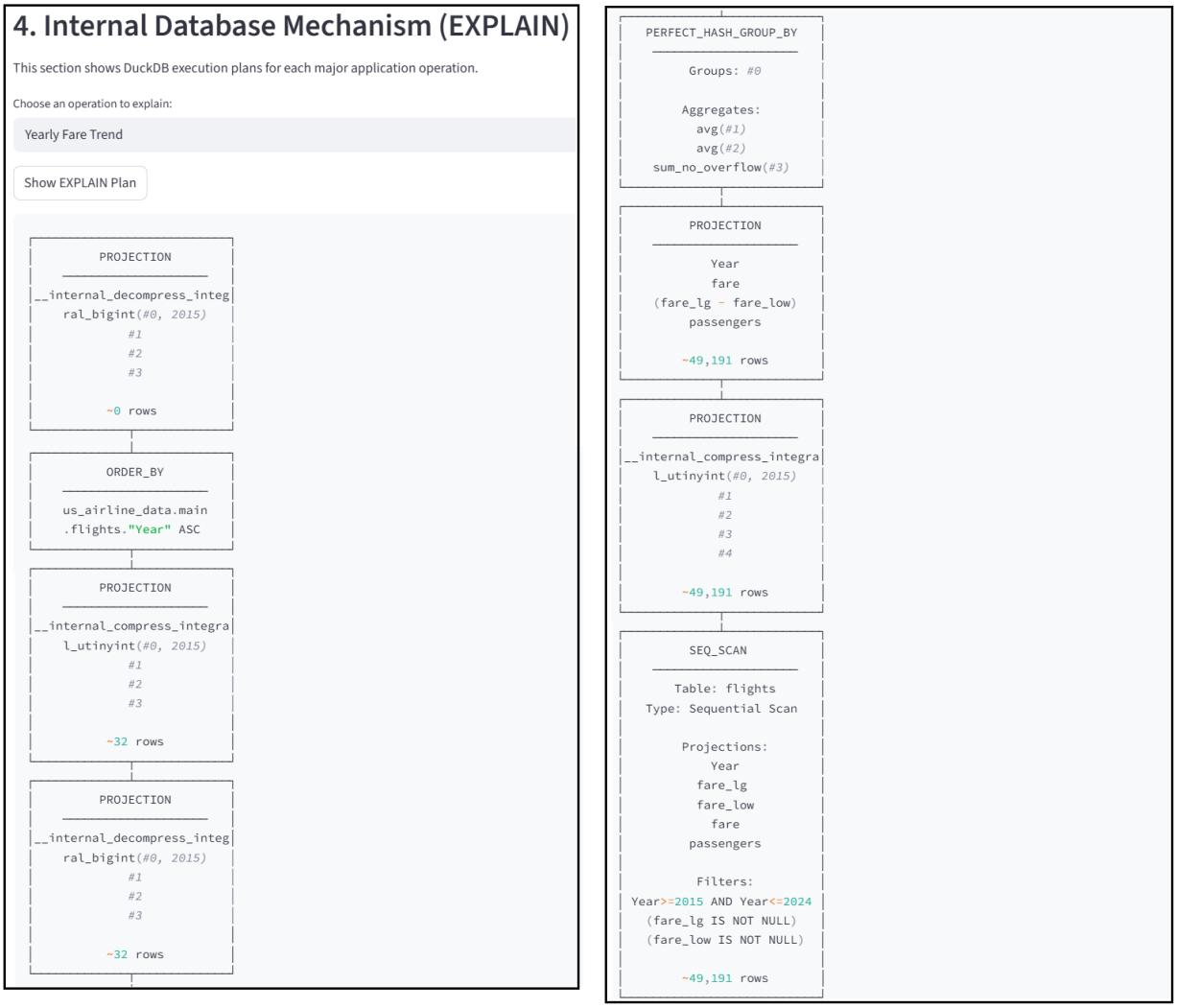


Figure B1: Execution plan for yearly fare trend query showing column scan and aggregation.

Appendix C — EXPLAIN: Route Market Summary

Figure C1: Execution plan showing route-level grouping and aggregation.

Appendix D — EXPLAIN: High-Concentration Routes

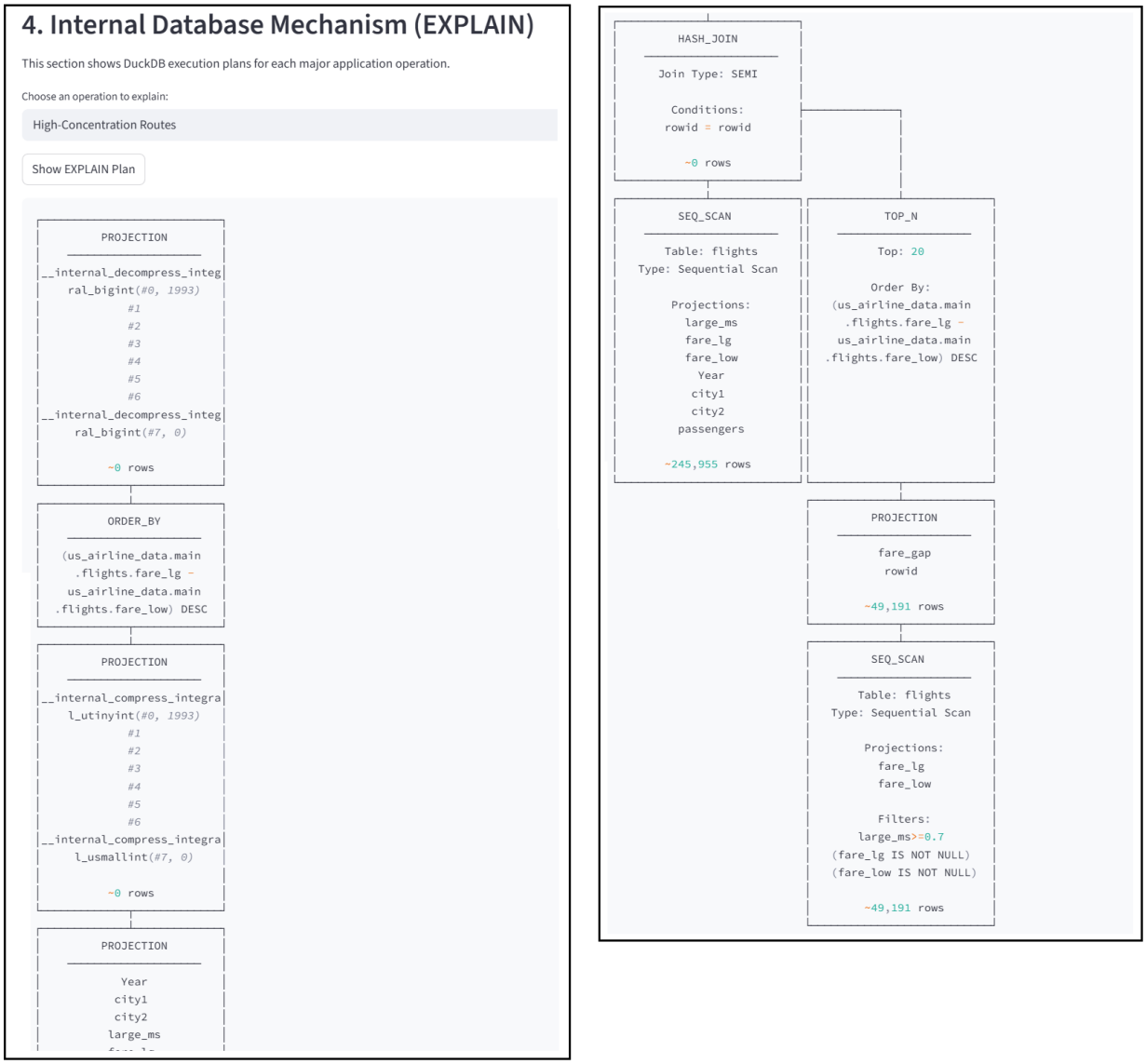
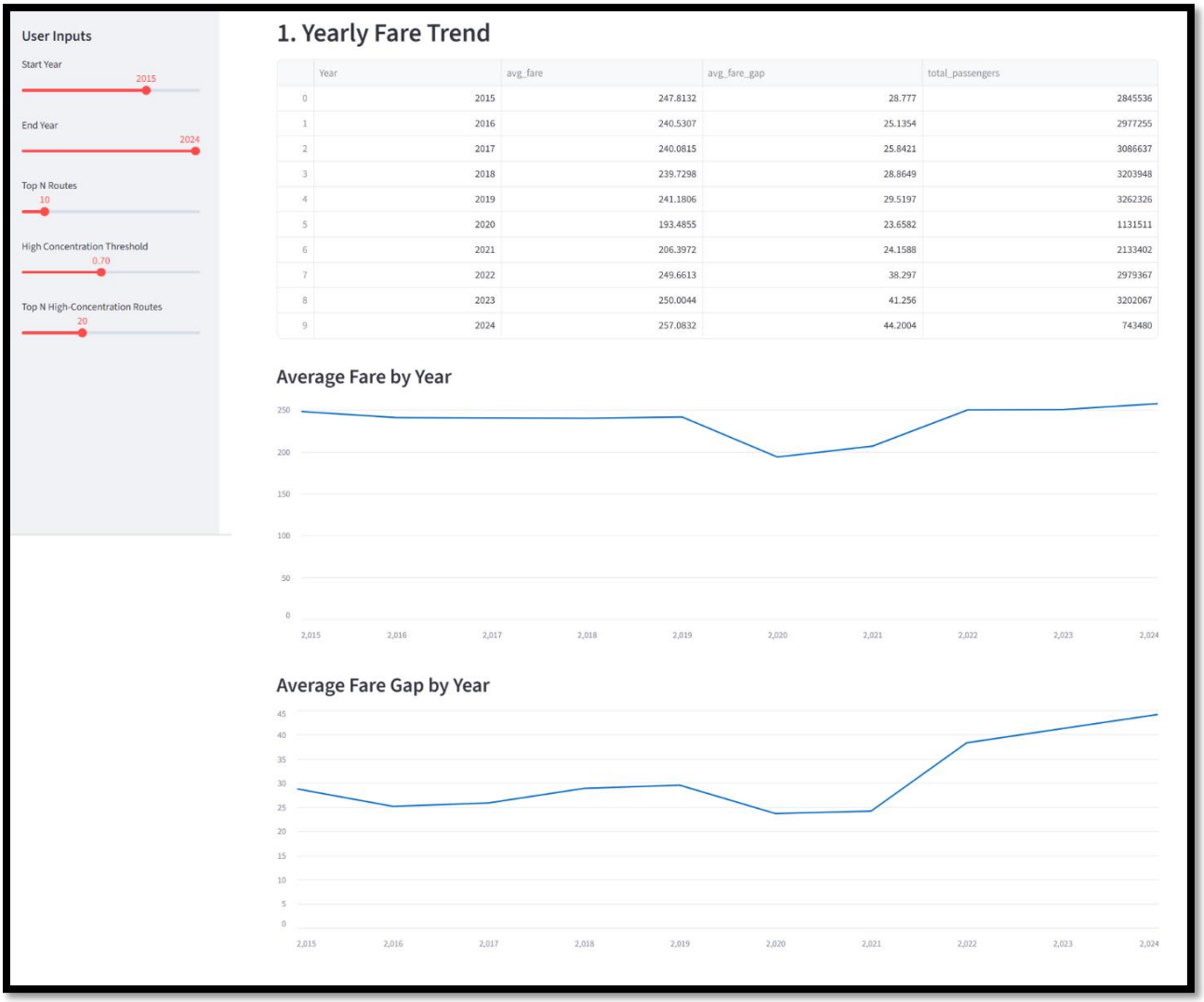


Figure D1: Execution plan demonstrating Top-N optimization and semi join.

Appendix E - Sample Outputs



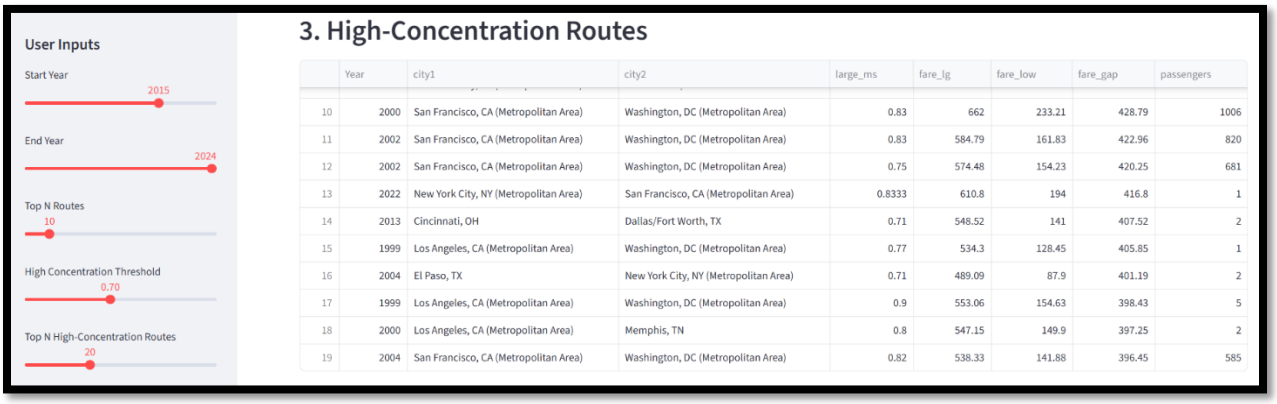
	Year	avg_fare	avg_fare_gap	total_passengers
0	2015	247.8132	28.777	2845536
1	2016	240.5307	25.1354	2977255
2	2017	240.0815	25.8421	3086637
3	2018	239.7298	28.8649	3203948
4	2019	241.1806	29.5197	3262326
5	2020	193.4855	23.6582	1131511
6	2021	206.3972	24.1588	2133402
7	2022	249.6613	38.297	2979367
8	2023	250.0044	41.256	3202067
9	2024	257.0832	44.2004	743480

Figure E1: Yearly fare trend output showing average fare and passenger trends over time.

2. Route Market Summary

	city1	city2	avg_market_share	avg_fare_gap	total_passengers
0	Los Angeles, CA (Metropolitan Area)	San Francisco, CA (Metropolitan Area)	0.8426	5.7157	2145874
1	Miami, FL (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6605	20.2187	1610441
2	Los Angeles, CA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6044	44.2548	1173011
3	New York City, NY (Metropolitan Area)	Orlando, FL	0.6714	15.3595	1034794
4	Chicago, IL	New York City, NY (Metropolitan Area)	0.723	13.1404	1019089
5	New York City, NY (Metropolitan Area)	San Francisco, CA (Metropolitan Area)	0.5935	51.5141	833969
6	Boston, MA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.7989	14.6611	760250
7	Atlanta, GA (Metropolitan Area)	New York City, NY (Metropolitan Area)	0.6986	38.4092	742844
8	Los Angeles, CA (Metropolitan Area)	Seattle, WA	0.7188	9.5703	696162
9	Las Vegas, NV	Los Angeles, CA (Metropolitan Area)	0.8149	8.5399	687331

Figure E2: Route market summary output showing route-level passenger volume, market share, and fare gap.



3. High-Concentration Routes

	Year	city1	city2	large_ms	fare_lg	fare_low	fare_gap	passengers
10	2000	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.83	662	233.21	428.79	1006
11	2002	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.83	584.79	161.83	422.96	820
12	2002	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.75	574.48	154.23	420.25	681
13	2022	New York City, NY (Metropolitan Area)	San Francisco, CA (Metropolitan Area)	0.8333	610.8	194	416.8	1
14	2013	Cincinnati, OH	Dallas/Fort Worth, TX	0.71	548.52	141	407.52	2
15	1999	Los Angeles, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.77	534.3	128.45	405.85	1
16	2004	El Paso, TX	New York City, NY (Metropolitan Area)	0.71	489.09	87.9	401.19	2
17	1999	Los Angeles, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.9	553.06	154.63	398.43	5
18	2000	Los Angeles, CA (Metropolitan Area)	Memphis, TN	0.8	547.15	149.9	397.25	2
19	2004	San Francisco, CA (Metropolitan Area)	Washington, DC (Metropolitan Area)	0.82	538.33	141.88	396.45	585

Figure E3: High-concentration routes output showing fare gaps and dominant carrier market share.